

MVLU COLLEGE

AI PRACTICAL NO. 4

AIM :- K-Nearest Neighbors (K-NN)

- Implement the K-NN algorithm for classification or regression.
- Apply the K-NN algorithm to a given dataset and predict the class or value for test data.
- Evaluate the accuracy or error of the predictions and analyze the results.

#CODE:

```
# PRACTICAL 4: K-NEAREST NEIGHBORS (K-NN)
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    mean_squared_error,
    r2_score
)
# Upload dataset
uploaded = files.upload()

# PART A - K NEAREST NEIGHBORS (K-NN) CLASSIFICATION

print("\n" + "=" * 60)
print("K-NEAREST NEIGHBORS (K-NN) CLASSIFICATION")
print("=" * 60)

# Target
y = df["Policy_Strength"]

# Features
X = df.drop(columns=["Policy_Strength"])

# Identify columns
cat_cols = X.select_dtypes(include="object").columns
```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
num_cols = X.select_dtypes(exclude="object").columns

# Preprocessing
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Find best K
k_values = range(1, 16)
accuracy_values = []

for k in k_values:

    model = Pipeline([
        ("preprocessor", preprocessor),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])

    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    accuracy_values.append(
        accuracy_score(y_test, prediction)
    )

# Best K
best_k = list(k_values)[
    accuracy_values.index(max(accuracy_values))
]
```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
print("\nBest K:", best_k)
print(
    "Best Accuracy:",
    round(max(accuracy_values) * 100, 2),
    "%"
)

# Final KNN Classification Model
knn_classifier = Pipeline([
    ("preprocessor", preprocessor),
    ("knn", KNeighborsClassifier(n_neighbors=best_k))
])

knn_classifier.fit(X_train, y_train)

y_pred = knn_classifier.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nKNN Classification Accuracy:")
print(round(accuracy * 100, 2), "%")

# KNN CONFUSION MATRIX

print("\nKNN CONFUSION MATRIX")

cm = confusion_matrix(y_test, y_pred)

print(cm)

plt.figure(figsize=(6, 5))

plt.imshow(cm)

plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
plt.colorbar()

classes = sorted(y.unique())

plt.xticks(range(len(classes)), classes)
plt.yticks(range(len(classes)), classes)

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.show()

# CLASSIFICATION ACTUAL VS PREDICTED

classification_result = pd.DataFrame({
    "Actual Class": y_test.values,
    "Predicted Class": y_pred
})

print("\nKNN CLASSIFICATION - ACTUAL VS PREDICTED")

print(classification_result.head(15))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# KNN CLASSIFICATION - K VS ACCURACY

plt.figure(figsize=(8, 5))
```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
plt.plot(  
    list(k_values),  
    accuracy_values,  
    marker="o"  
)  
  
plt.xlabel("K Value")  
plt.ylabel("Accuracy")  
plt.title("KNN Classification: K vs Accuracy")  
  
plt.grid()  
plt.show()  
  
# PART B - K NEAREST NEIGHBORS (K-NN) REGRESSION  
  
print("\n" + "=" * 60)  
print("K-NEAREST NEIGHBORS (K-NN) REGRESSION")  
print("=" * 60)  
  
# Target  
y_reg = df["Plastic_Tons"]  
  
# Features  
X_reg = df.drop(columns=["Plastic_Tons"])  
  
# Identify columns  
cat_cols_reg = X_reg.select_dtypes(  
    include="object"  
) .columns  
  
num_cols_reg = X_reg.select_dtypes(  
    exclude="object"  
) .columns  
  
# Preprocessing  
preprocessor_reg = ColumnTransformer([  
    (  
        "num",  
        StandardScaler(),  
        num_cols_reg,  
    ),  
    (  
        "cat",  
        OneHotEncoder(),  
        cat_cols_reg,  
    ),  

```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
        num_cols_reg
    ),
    (
        "cat",
        OneHotEncoder(handle_unknown="ignore"),
        cat_cols_reg
    )
])

# Train-test split
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.25,
    random_state=42
)

# TEST DIFFERENT K VALUES

k_values_reg = range(1, 16)

r2_values = []
rmse_values = []

for k in k_values_reg:

    model = Pipeline([
        ("preprocessor", preprocessor_reg),
        (
            "knn",
            KNeighborsRegressor(
                n_neighbors=k
            )
        )
    ])

    model.fit(
        X_train_reg,
        y_train_reg
```

MVLU COLLEGE AI PRACTICAL NO. 4

```
)

prediction = model.predict(
    X_test_reg
)

# R2
r2 = r2_score(
    y_test_reg,
    prediction
)

# RMSE
rmse = mean_squared_error(
    y_test_reg,
    prediction
) ** 0.5

r2_values.append(r2)
rmse_values.append(rmse)

# Best K according to R2
best_k_reg = list(k_values_reg)[
    r2_values.index(max(r2_values))
]

print("\nBest K for Regression:")
print(best_k_reg)

print("\nBest R2 Score:")
print(round(max(r2_values), 4))

print("\nLowest RMSE:")
print(round(min(rmse_values), 2))

# FINAL KNN REGRESSION MODEL

knn_regressor = Pipeline([
```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
("preprocessor", preprocessor_reg),
(
    "knn",
    KNeighborsRegressor(
        n_neighbors=best_k_reg
    )
)
])

knn_regressor.fit(
    X_train_reg,
    y_train_reg
)

y_pred_reg = knn_regressor.predict(
    X_test_reg
)

# REGRESSION RESULTS

r2_final = r2_score(
    y_test_reg,
    y_pred_reg
)

rmse_final = mean_squared_error(
    y_test_reg,
    y_pred_reg
) ** 0.5

print("\nKNN REGRESSION RESULTS")

print("R2 Score:", round(r2_final, 4))
print("RMSE:", round(rmse_final, 2))

# KNN REGRESSION ACTUAL VS PREDICTED

regression_result = pd.DataFrame({
```

MVLU COLLEGE

AI PRACTICAL NO. 4

```
"Actual Plastic Tons": y_test_reg.values,
"Predicted Plastic Tons": y_pred_reg.round(2)
})

print("\nKNN REGRESSION ACTUAL VS PREDICTED")

print(regression_result.head(15))

# ACTUAL VS PREDICTED VALUES GRAPH

plt.figure(figsize=(8, 5))

plt.scatter(
    y_test_reg,
    y_pred_reg
)

plt.xlabel("Actual Plastic Tons")
plt.ylabel("Predicted Plastic Tons")

plt.title(
    "KNN Regression: Actual vs Predicted Values"
)

plt.grid()

plt.show()

# KNN REGRESSION: R2 SCORE VS K

plt.figure(figsize=(8, 5))

plt.plot(
    list(k_values_reg),
    r2_values,
    marker="o"
)
```

MVLU COLLEGE AI PRACTICAL NO. 4

```
plt.xlabel("K Value")
plt.ylabel("R2 Score")

plt.title(
    "KNN Regression: R2 Score vs K"
)

plt.grid()

plt.show()

# KNN REGRESSION: RMSE VS K

plt.figure(figsize=(8, 5))

plt.plot(
    list(k_values_reg),
    rmse_values,
    marker="o"
)

plt.xlabel("K Value")
plt.ylabel("RMSE")

plt.title(
    "KNN Regression: RMSE vs K"
)

plt.grid()

plt.show()

# FINAL OUTPUT

print("\n" + "=" * 60)
print("FINAL KNN RESULTS")
print("=" * 60)
```

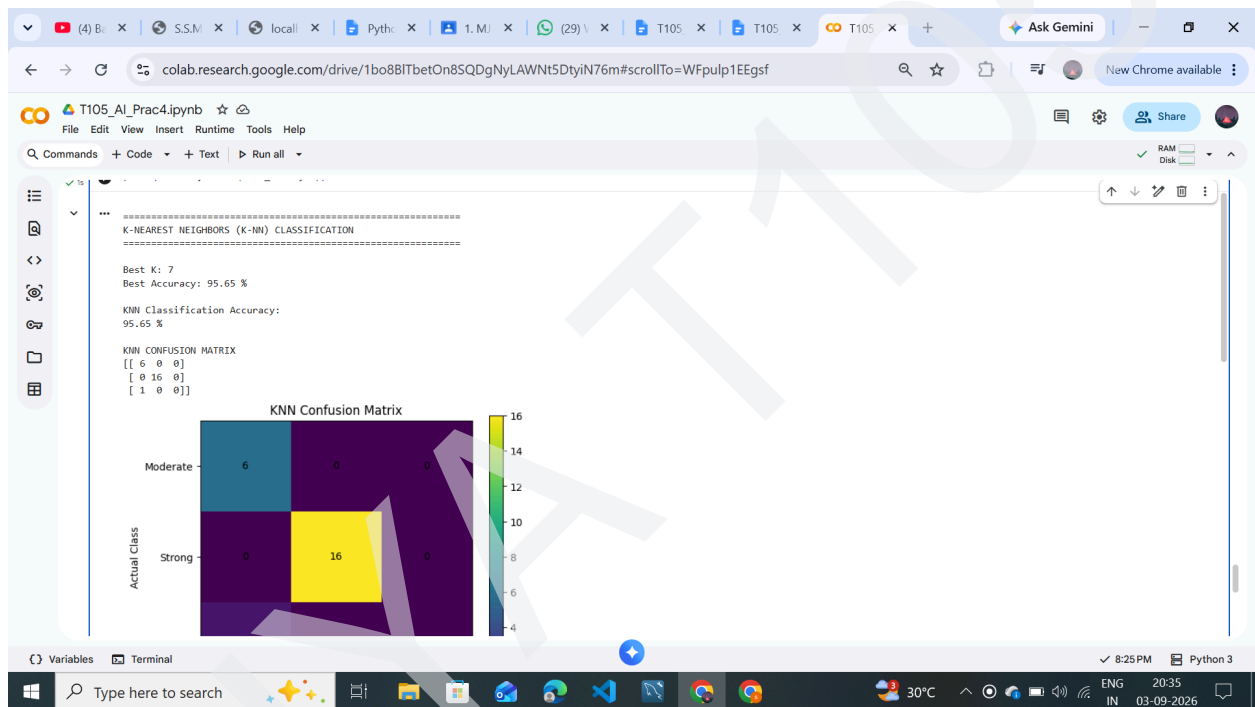
MVLU COLLEGE

AI PRACTICAL NO. 4

```
print("\nCLASSIFICATION")
print("Best K:", best_k)
print("Accuracy:", round(accuracy * 100, 2), "%")

print("\nREGRESSION")
print("Best K:", best_k_reg)
print("R2 Score:", round(r2_final, 4))
print("RMSE:", round(rmse_final, 2))
```

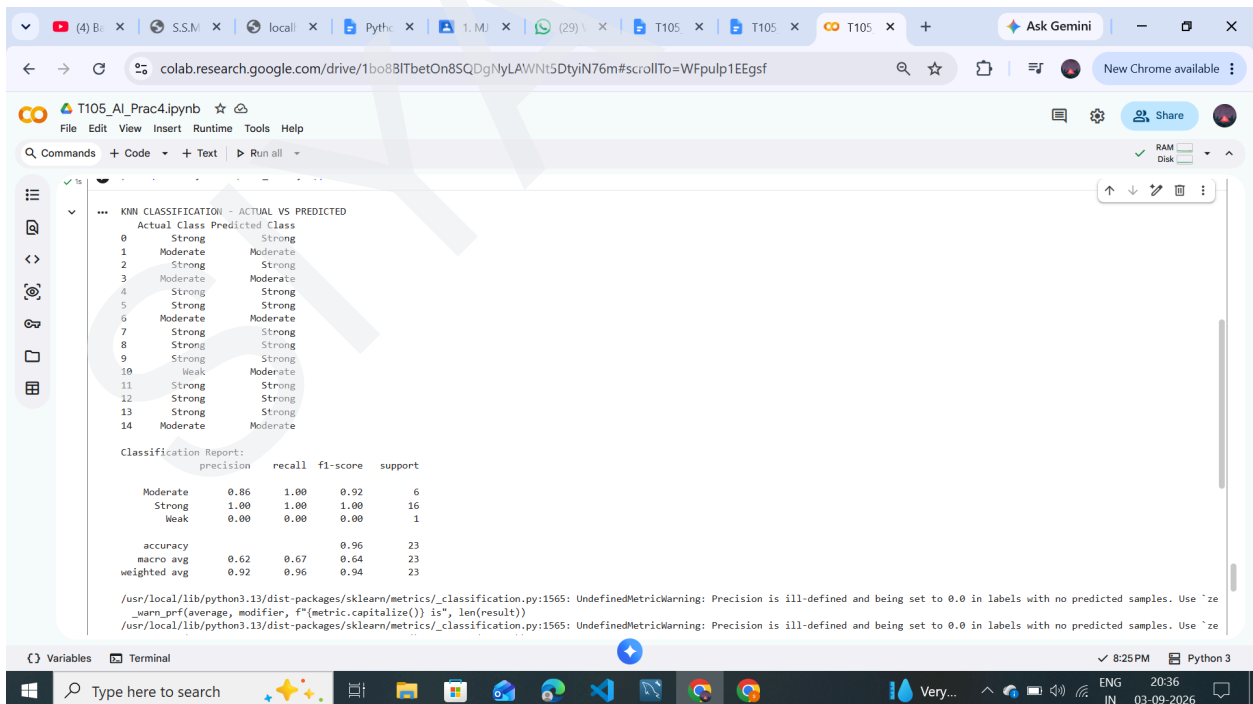
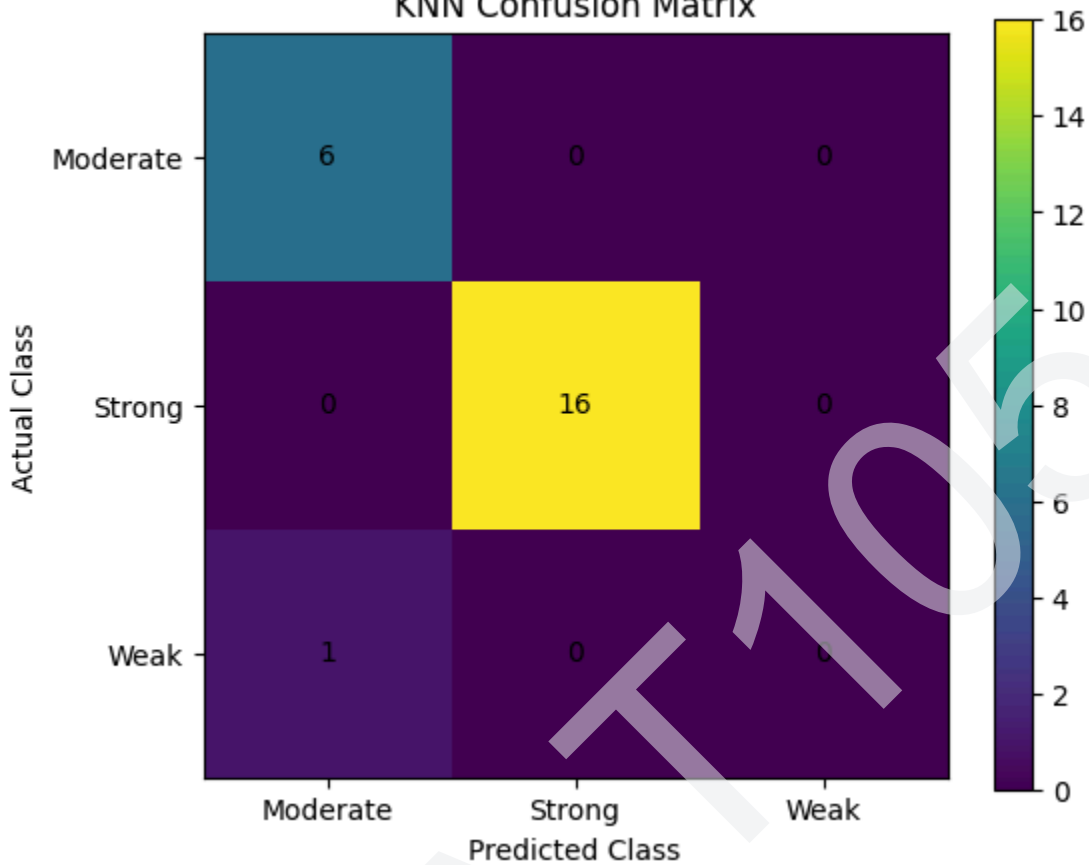
#OUTPUT:



MVLU COLLEGE

AI PRACTICAL NO. 4

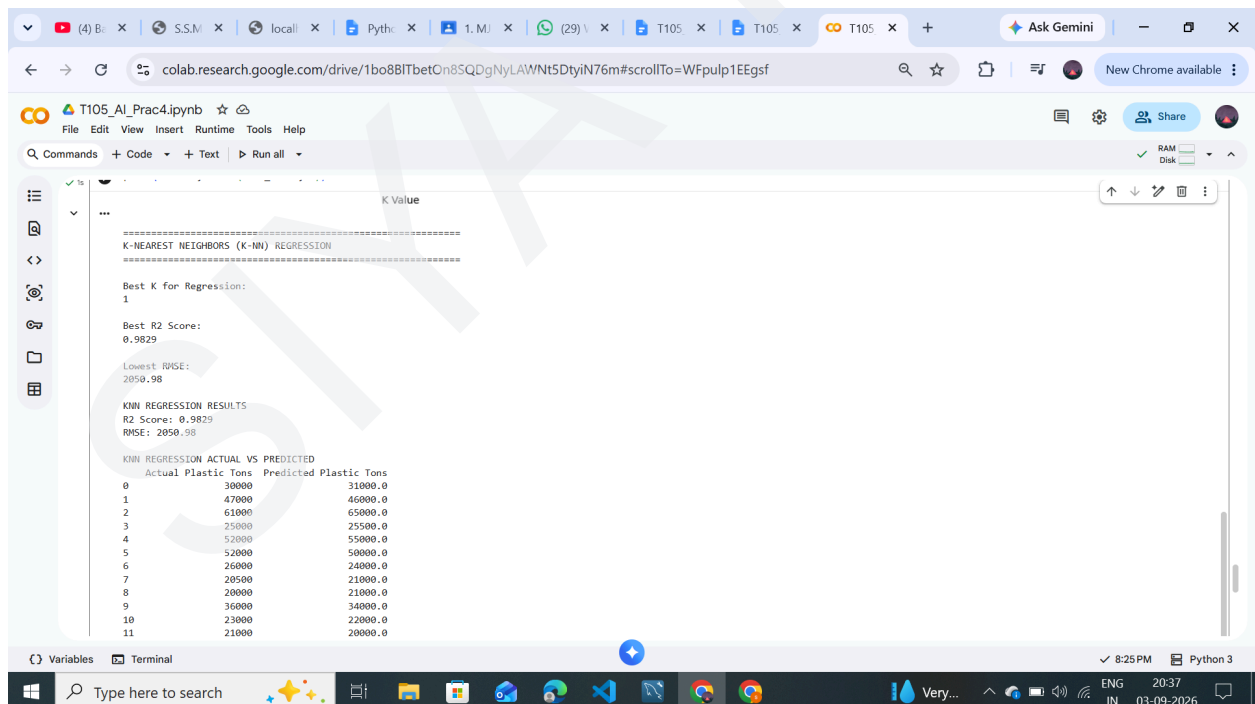
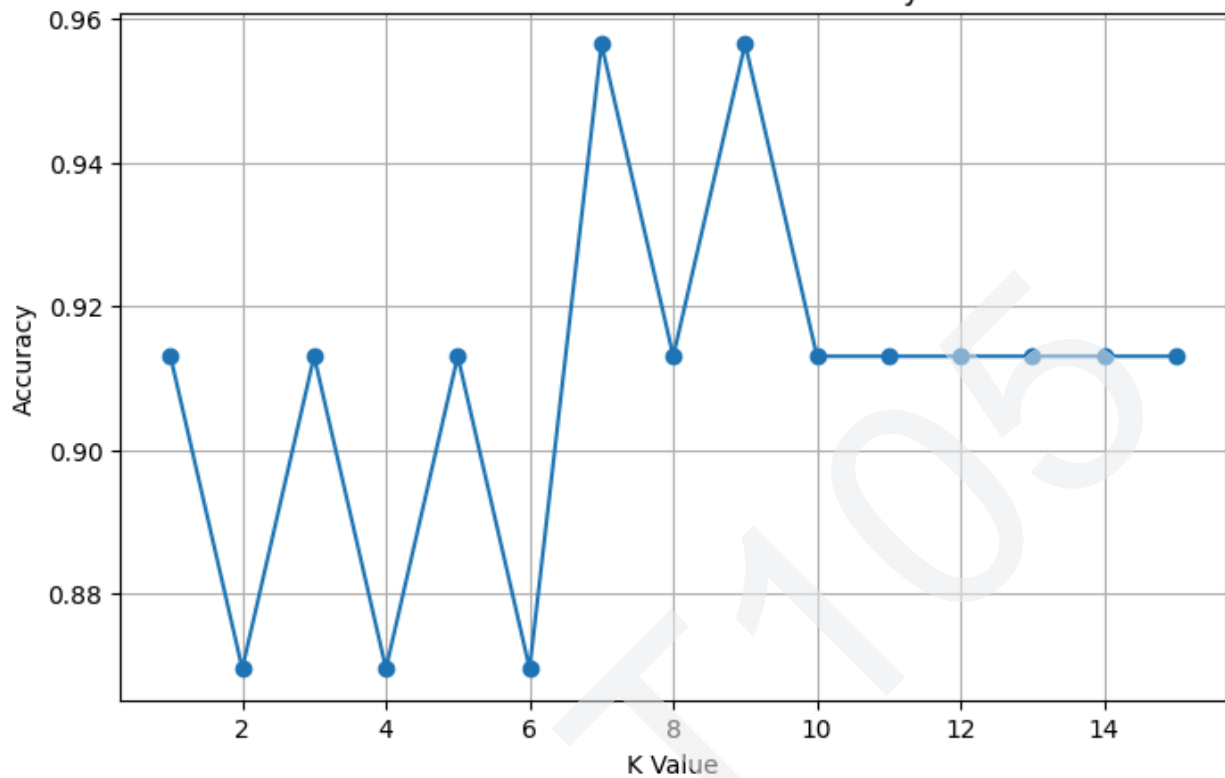
KNN Confusion Matrix



MVLU COLLEGE

AI PRACTICAL NO. 4

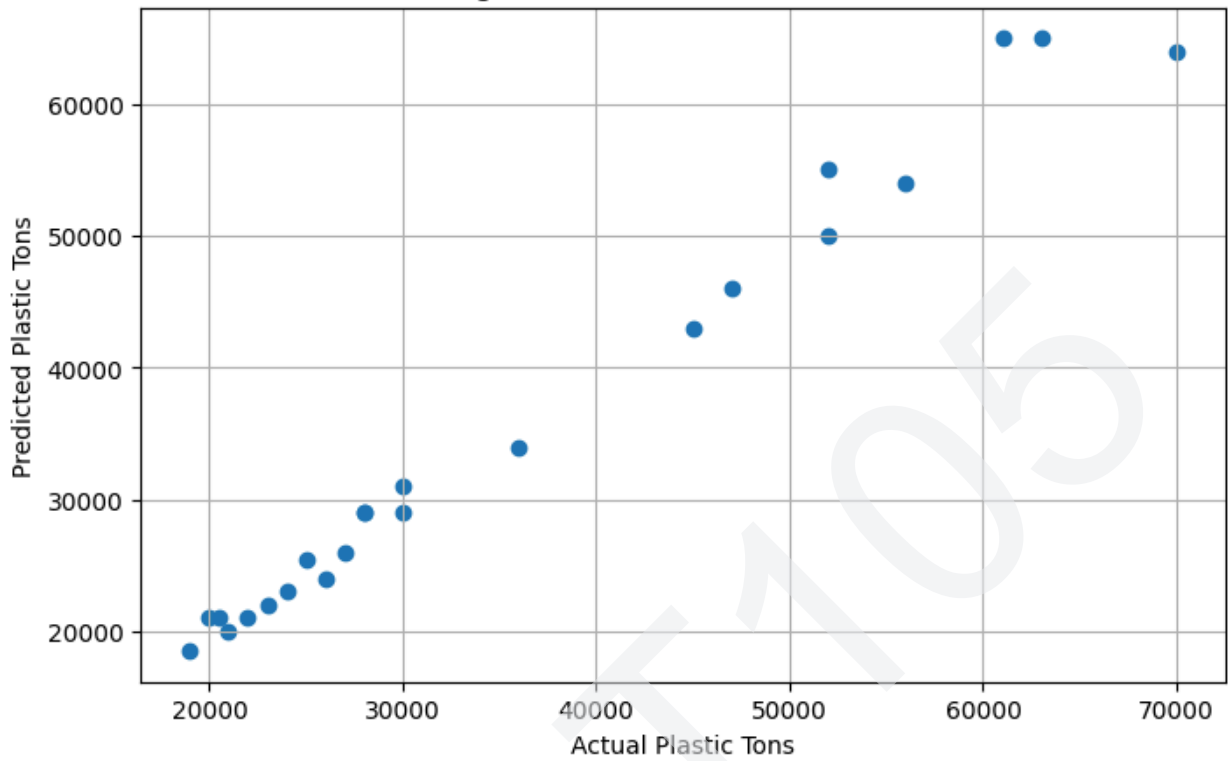
KNN Classification: K vs Accuracy



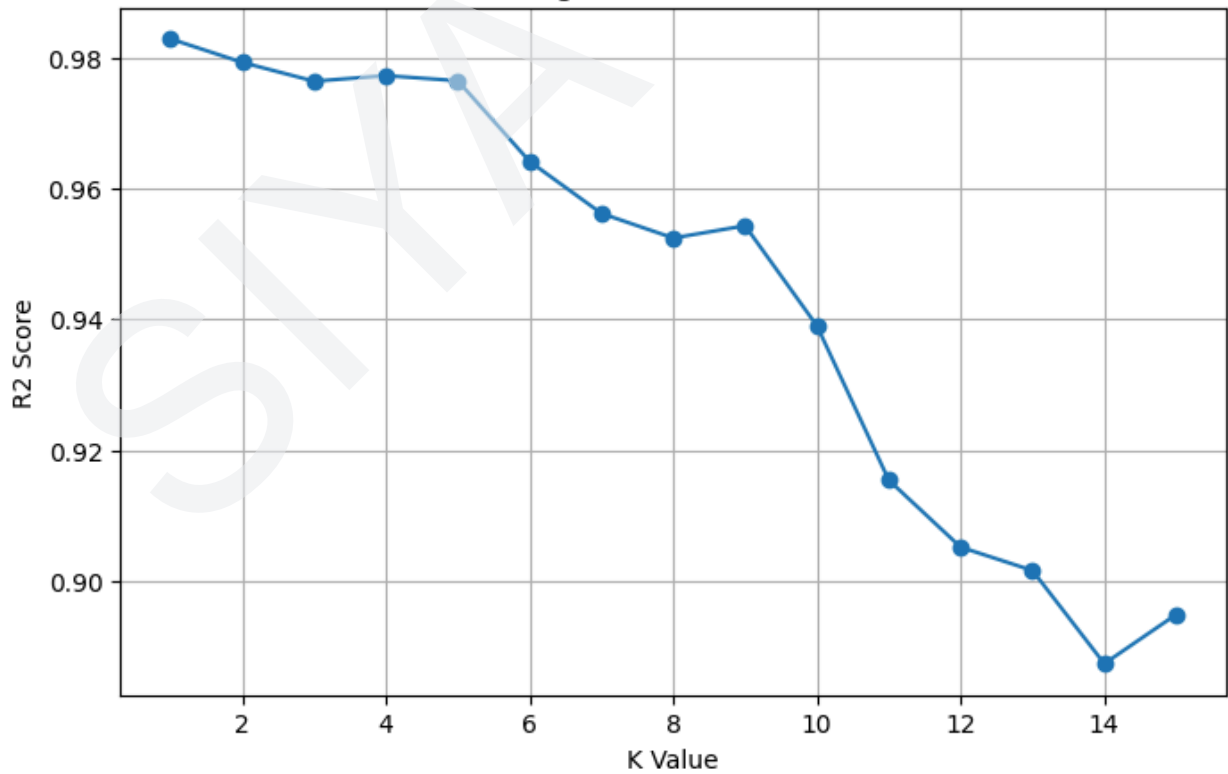
MVLU COLLEGE

AI PRACTICAL NO. 4

KNN Regression: Actual vs Predicted Values



KNN Regression: R2 Score vs K



MVLU COLLEGE

AI PRACTICAL NO. 4

KNN Regression: RMSE vs K

